Department of Mechanical and Production Engineering
Islamic University of Technology

ME 4879: Robotics

Instructor: Dr. Kazi Arafat Rahman, ME, BUET

Course Project 2

Modeling, Stability, and Control of a Quadrotor

Overview

Building on Lectures 5 and 6, this project takes a quadrotor from first principles to simulated flight. You will specify a vehicle and its hardware, derive its nonlinear dynamics, cast them in state-space form, analyze the stability of hover, and design a feedback controller that flies the machine along a desired trajectory. A short Python hint accompanies each section, pointing to the tools you will need; the modeling decisions and the code remain yours.

1. Vehicle Specification and Bill of Quantities

- Choose either the *planar* quadrotor (the vehicle studied in lecture) or the full *3D* quadrotor; the latter is more ambitious.
- Fix the physical parameters your model needs: mass m, arm length L, moment(s) of inertia, the thrust and rotor-torque coefficients, and the per-motor thrust limit.
- Prepare a **Bill of Quantities** for the components that would physically realize this vehicle—frame, brushless motors, ESCs, propellers, flight controller / IMU, battery, power-distribution board, radio link—with quantities and indicative specifications, and connect each item back to a model parameter.

Python hint. Keep every physical parameter in one place, a `dict` or a `@dataclass`, so all later functions read from it. Build the Bill of Quantities as a `pandas.DataFrame` and export it with `df.to_latex()` or `df.to_markdown()`.

2. Quadrotor Dynamics

- Derive the equations of motion (Newton–Euler). Define the state vector and the body frame $(\hat{b}_1, \hat{b}_2, \hat{b}_3)$ used in lecture, with thrust along $\hat{b}_3$, and give the rotation matrix $\mathbf{R}$ mapping body-frame vectors to inertial coordinates.
- Use the lecture's control parameterization: collective thrust together with the body moments, rather than the four individual rotor thrusts, and state the linear map between the two.
- Assemble the nonlinear state-space model $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$.

Python hint. Derive the equations symbolically with `sympy` (`sympy.Matrix`, `.diff()`), then make them fast with `sympy.lambdify` into a function `f(t, x, u, p)` returning `xdot` as a NumPy array. Form $\mathbf{R}$ with `scipy.spatial.transform.Rotation`. The thrust-to-moment mixer is a fixed matrix; invert it once with `numpy.linalg.inv`.

3. Stability

- Identify the hover equilibrium: the nominal control $\mathbf{u}_0$ that supplies thrust mg and zero moment at the reference state.

- Linearize about hover to obtain $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$. Examine open-loop stability through the eigenvalues of $\mathbf{A}$, and comment on the unstable / marginal modes that arise because the quadrotor is underactuated.
- Verify that the linearized system is controllable (stabilizable) before attempting control design.

Python hint. Linearize either symbolically (`Matrix.jacobian` of f with respect to $\mathbf{x}$ and $\mathbf{u}$, evaluated at hover) or numerically by finite differences. Read the open-loop modes with `numpy.linalg.eigvals(A)`, and test controllability with `control.ctrb(A, B)` followed by `numpy.linalg.matrix_rank`.

4. Control: LQR / PID

- Design a stabilizing feedback controller of the form $\mathbf{u} = \mathbf{u}_0 - \mathbf{K}(\mathbf{x} - \mathbf{x}_0)$. Use an **LQR** (choosing $\mathbf{Q}$ and $\mathbf{R}$ and solving for $\mathbf{K}$) and/or a **PID/PD** controller; if you implement both, compare transient response, steady-state error, and control effort.
- Demonstrate that the controller stabilizes hover from a perturbed initial state.
- **Fly the quadrotor:** command it to track a trajectory or a sequence of waypoints (step setpoints suffice; a smooth flat-output trajectory in the sense of Lecture 6 is the more ambitious option).

Python hint. Solve the LQR in one line with `K, S, E = control.lqr(A, B, Q, R)` (or `scipy.linalg.solve_continuous_are`). Start from `Q = np.diag([...])` and `R = np.eye(...)`, then reweight to trade tracking against effort. A PID loop is a few lines accumulating the state error each step.

5. Simulation and Flight

- Implement the model and controller in Python, integrating the *nonlinear* dynamics with the controller in the loop.
- Report the state trajectories, the control inputs, and the tracking error, and produce a short animation or set of rendered frames of the quadrotor in flight.

Python hint. Integrate the closed loop with `scipy.integrate.solve_ivp(closed_loop, [0, T], x0, t_eval=...)`. Plot with `matplotlib`; build the animation with `matplotlib.animation.FuncAnimation` (use `mpl_toolkits.mplot3d` for the 3D case) and save via `anim.save("flight.mp4")`.

6. Deliverables

- A concise technical report containing: the Bill of Quantities; the dynamics derivation and state-space model; the hover linearization with the open-loop stability and controllability analysis; the controller design with the chosen gains or cost matrices; the simulation results; and a short discussion.
- Supporting artifacts: your code and the flight animation / rendered frames. The project may be undertaken individually or in small groups.

Python hint. Keep the parameter dictionary at the top of a single script so the run is reproducible; export figures with `fig.savefig(...)` and the animation with `anim.save(...)` for inclusion in the report.

Notation. Use the conventions of Lectures 5–6: the body frame $(\hat{b}_1, \hat{b}_2, \hat{b}_3)$ with thrust along $\hat{b}_3$; the thrust-and-moment control parameterization; the state vector and rotation matrix $\mathbf{R}$ as defined in lecture; and the feedback law $\mathbf{u} = \mathbf{u}_0 - \mathbf{K}(\mathbf{x} - \mathbf{x}_0)$.